QUALITY ASSURANCE · MOBILE TEST AUTOMATION

SalesHiker Mobile QA Automation Framework

SalesHiker Android app - how the automated testing works, what it covers, and where we stand today

Application	SalesHiker for Android (`com.saleshiker.app 1.0.0`); beta backend `accountsbeta.saleshiker.com` ; test account in `config\.env`
Test device	RMX3997, Android 16 (real phone over USB); emulator scripts kept for Android Virtual Devices
Status as of	5 November 2026
Prepared by	KalpeshMevada03062019, QA (generated from the repository)
Source	Repository `KalpeshMevada03062019/SalesHikerMobileApp` , branch `main` (commit fd33695)

Numbers in this document come from the test specs, the results ledger, the test-case and defect workbooks and the kept run reports on 5 November 2026. They change with every run; the document is rebuilt automatically after each test run (`npm run docs:framework`).

1. Summary

We test the **SalesHiker Android app** with one automated framework. It drives the real app on a real phone the way a user would (taps, typing, scrolling), checks what the screen shows, and records evidence (screenshots, UI hierarchy, device logs) for every key step and every failure.

202	18	22	1
automated tests (18 smoke)	modules (56 spec files)	defects logged (22 open)	device: RMX3997, Android 16

What the framework gives the team

- **Readable tests:** every test has an ID (e.g. `MCMP-012`) and a plain title, so anyone can read what is covered and run exactly that test.
- **Defects the team can act on:** each defect has steps to reproduce, expected and actual result, severity and screenshots, kept in the Excel defect tracker.
- **Automatic records:** after every run, results go into the test-case workbook, failures without a defect go to a triage sheet, defect status moves when its guard test changes, and this document is rebuilt.
- **Safe on a live workspace:** real WhatsApp messages reach only 2 allowlisted test numbers, only when switched on, and the recipients are checked before and after sending.

Status today

Area	Status
AI Agents	Built and run: 12 PASS, 0 FAIL last recorded (3 Nov 2026); 1 defect logged
Broadcasts (WhatsApp)	Built and run: 12 PASS, 1 FAIL last recorded (2 Nov 2026); 2 defects logged
Calendar (CRM)	Built and run: 7 PASS, 0 FAIL last recorded (4 Nov 2026 – 5 Nov 2026)
Campaigns (WhatsApp)	Built and run: 17 PASS, 2 FAIL last recorded (30 Oct 2026); 3 defects logged
Chatbots (WhatsApp)	Built and run: 8 PASS, 0 FAIL last recorded (3 Nov 2026)
Documents (CRM)	Built and run: 7 PASS, 0 FAIL last recorded (5 Nov 2026)
Inbox (WhatsApp chats)	Built and run: 12 PASS, 3 FAIL last recorded (2 Nov 2026); 4 defects logged
Leads (CRM)	Built and run: 8 PASS, 1 FAIL last recorded (4 Nov 2026); 1 defect logged
Login (mobile sign-in)	Built and run: 23 PASS, 1 FAIL last recorded (2 Nov 2026); 4 defects logged
Navigation (before sign-in)	Built and run: 2 PASS, 0 FAIL last recorded (2 Nov 2026)
Organizations (CRM)	Built and run: 8 PASS, 0 FAIL last recorded (4 Nov 2026)
Pipeline (Deals)	Built and run: 10 PASS, 2 FAIL last recorded (2 Nov 2026); 3 defects logged
Quotations (Sales)	Built and run: 8 PASS, 1 FAIL last recorded (5 Nov 2026); 1 defect logged
Tasks (CRM)	Built and run: 14 PASS, 1 FAIL last recorded (4 Nov 2026); 1 defect logged
Templates (WhatsApp)	Built and run: 14 PASS, 1 FAIL last recorded (3 Nov 2026); 1 defect logged
Tickets (CRM)	Built and run: 7 PASS, 1 FAIL last recorded (5 Nov 2026); 1 defect logged
WhatsApp Logs	Built and run: 10 PASS, 0 FAIL last recorded (3 Nov 2026)
Workflows (CRM)	Built and run: 9 PASS, 0 FAIL last recorded (3 Nov 2026 – 4 Nov 2026)
Nightly / on-push run (CI)	**Not configured**: tests need a USB-connected phone, so they run on the QA machine only.

Needs attention: (1) open High defects MINB-D01.

2. What the framework is built with

Purpose	Tool	Why
Mobile automation	**Appium** 3.9.0 with the **UiAutomator2** driver 8.7.2	Drives the native Android app through the accessibility tree; no changes to the app are needed.
Test runner	**WebdriverIO** 10.1.0 with **Mocha** (@wdio/mocha-framework 10.1.0) and **Chai** 6.3.0	Starts Appium, runs one spec per session, waits for elements, readable assertions.
Language	**TypeScript** 7.0.2, Node.js 24.19.0	Class-based page objects and typed helpers; errors caught before a run.
Device access	Android SDK platform-tools (adb 37.0.1), no Android Studio	Installs the APK, reads device logs and ANR events, captures screenshots.
Reports	Allure 2.46.1 (single-file HTML), JUnit, `summary.md`	Readable results with screenshots per run; machine-readable results for the workbooks.
Test cases and defects	**exceljs** 4.4.0 + `tools/excel/update-workbooks.js`	Writes run results into the test-case workbook and keeps the defect tracker up to date.
Configuration	dotenv 18.0.9	Device, test account and send switches live in a local `config\.env` (never in git).
Code quality	TypeScript strict mode, `npm run typecheck`	Type errors are caught before tests run.
Automation server	None	**Not configured**: tests need the USB phone on the QA machine.

How the code is organised

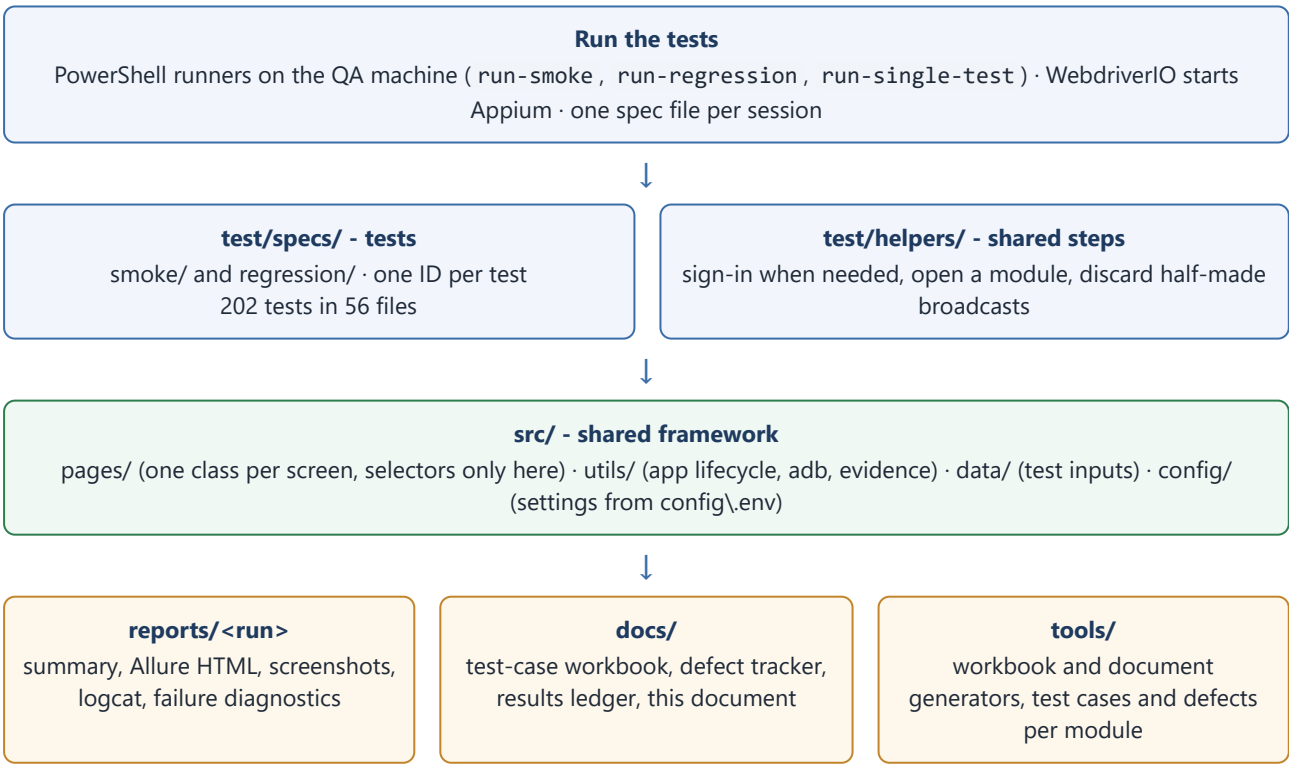


Figure 1 - Layers. Specs say *what* is tested, page classes know *how* to use each screen, and the shared framework handles the device, sign-in, evidence and safety for every module.

3. How it works

3.1 From a new module to tested coverage

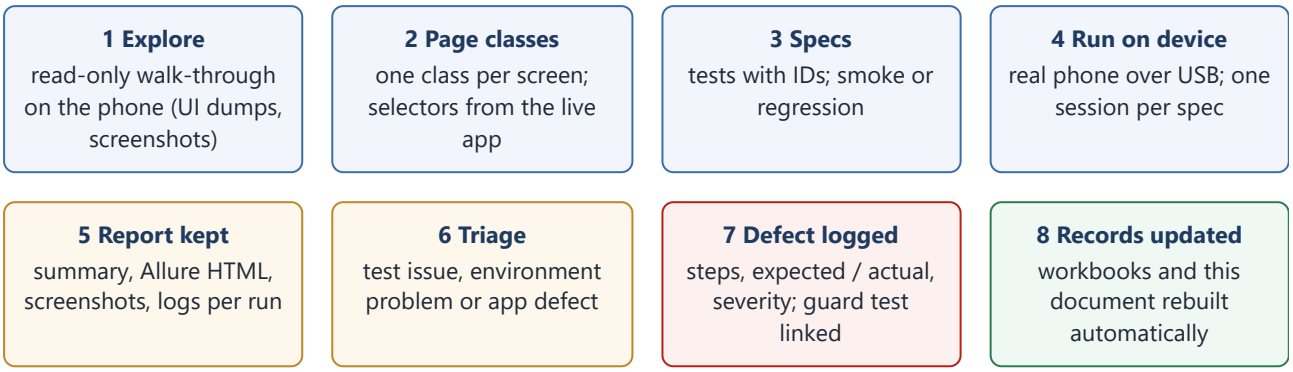


Figure 2 - Testing workflow. A failure first goes to the "Failures to triage" sheet; it becomes a defect only after triage. A test problem is fixed and re-run instead.

3.2 Defect lifecycle

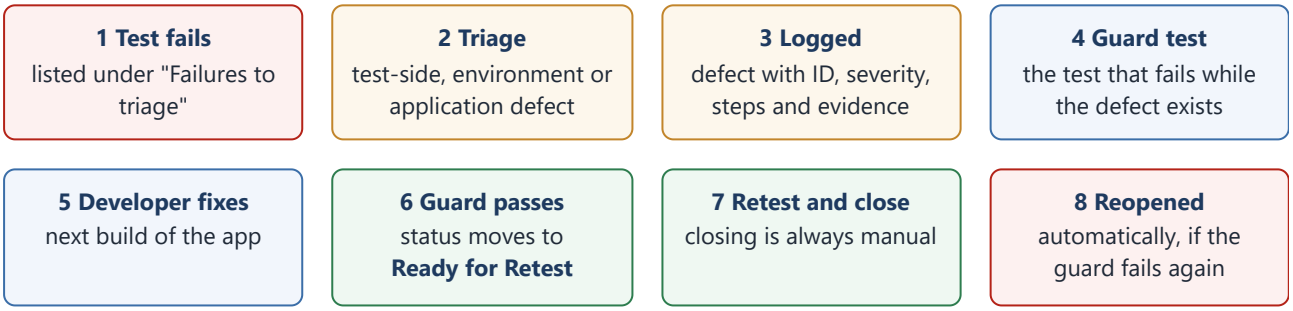


Figure 3 - A guard test keeps failing while its defect is open (its assertions describe the correct behaviour and are never relaxed). When it passes, the tracker moves the defect to Ready for Retest; "Needs Decision" and "Won't Fix" are never changed automatically.

3.3 What happens in one run

1. **Checks:** the runner confirms the phone is connected and the app is installed, and records its version.
2. **Session:** WebdriverIO starts Appium; the UiAutomator2 helper already on the phone is reused (ColorOS blocks reinstalls behind a security prompt).
3. **Each test** restarts the app; signed-in tests sign in only when the session is gone. Tests that need data the account lacks are **skipped with a reason**, never passed.
4. **Safety:** sending needs `BROADCAST_SEND=true`; the audience and recipients are checked against the allowlist; nothing is deleted.
5. **Evidence:** screenshots at key steps; on failure the screen, UI hierarchy, foreground app and logcat are saved.
6. **After the run:** environment aborts are reported as "not executed" (not failed), then the test-case workbook, defect tracker and this document are updated.

4. What we test, and how we keep the workspace safe

4.1 Kinds of tests

Kind	What it means	Tests
`smoke`	The essentials of each module (`test/specs/smoke`); run first	18
`regression`	Full behaviour per module (`test/specs/regression`)	184
`needs account`	Needs the beta test account from `config\.env`; skipped when it is missing	185
`expected to fail`	Guard tests of open defects: fail until the defect is fixed	15
`sends WhatsApp`	Creates and sends a real broadcast (allowlisted numbers only); stops before sending unless switched on	1
`data dependent`	Skipped with a reason when the account has no matching data (e.g. no campaign with replies)	29

Tests run by module with `-Grep <prefix>` (`MAIA` `MBRD` `MCAL` `MCMP` `MCBT` `MDOC` `MINB` `MLED` `MLGN` `REG-NAV` `MORG` `MPIP` `MQTN` `MTSK` `MTPL` `MTKT` `MWLG` `MWFL`) or one by one with its ID. There are no accessibility, performance, API, iOS or visual-comparison tests: **not covered**.

4.2 Safety rules on the live workspace

Rule	How it is enforced
Beta backend only	The APK under test points to `accountsbeta.saleshiker.com`; production is never used.
Real messages only to test numbers	The broadcast audience is a tag carried only by the 2 allowlisted contacts. Before sending, each number must have exactly one contact with that tag; after sending, the recipients must be exactly the allowlist.
Sending needs a switch	`BROADCAST_SEND=true` must be set; otherwise the send test runs every check and stops before "Send Now" (skipped).
Send once, never retry	"Send Now" is tapped once; "All contacts" can never be chosen by the automation.
No half-made broadcasts	Every wizard test ends with Cancel / Discard; a safety net discards an open wizard after each test.
Read-only elsewhere	Campaign tests never create, pause or edit campaigns; nothing is deleted.
No secrets in git	Credentials, device serial and allowlist live in `config\.env` (git-ignored); the runner removes the password from all reports.

5. Scope

In scope

Module	Tests	Smoke	Defects	Last recorded PASS / FAIL
AI Agents	12	1	1	12 / 0
Broadcasts (WhatsApp)	13	1	2	12 / 1
Calendar (CRM)	7	1	–	7 / 0
Campaigns (WhatsApp)	19	1	3	17 / 2
Chatbots (WhatsApp)	8	1	–	8 / 0
Documents (CRM)	7	1	–	7 / 0
Inbox (WhatsApp chats)	15	1	4	12 / 3
Leads (CRM)	9	1	1	8 / 1
Login (mobile sign-in)	24	2	4	23 / 1
Navigation (before sign-in)	2	0	–	2 / 0
Organizations (CRM)	8	1	–	8 / 0
Pipeline (Deals)	12	1	3	10 / 2
Quotations (Sales)	9	1	1	8 / 1
Tasks (CRM)	15	1	1	14 / 1
Templates (WhatsApp)	15	1	1	14 / 1
Tickets (CRM)	8	1	1	7 / 1
WhatsApp Logs	10	1	–	10 / 0
Workflows (CRM)	9	1	–	9 / 0
Total	**202**	**18**	**22**	**188 / 14**

Defects are the records in `docs/SalesHiker_Mobile_Defect_Tracker.xlsx` (open and closed). Results are the last PASS/FAIL per test in `docs/test-execution-ledger.json`; skipped and not-executed runs never replace a result.

Out of scope, or waiting

Item	Status / reason
iOS app	Not covered: Android only.
Other app areas	Not covered: Dashboard, Inbox (chats), Contacts and Deals tabs, Templates, Chatbots, AI Agents, WhatsApp Logs, Organizations, Leads, Tasks, Calendar.
Scheduled broadcasts, drafts	Not covered: only "Send now" is tested; scheduling would leave a pending send.
Emulator runs	Scripts kept, but the emulators on this PC are unstable under memory pressure; tests run on the real phone.
API / back-end tests	Out of scope: UI tests only.
Test environment	Tests run against the shared beta workspace; no dedicated test workspace.

6. Results snapshot - 5 November 2026

Defects by severity

Severity	Defects
Critical	0
High	1
Medium	6
Low	15
Total	**22**

Defects by status

Status	Defects
Needs Decision	10
Open	11
Ready for Retest	1

Last recorded test results: **188 PASS · 14 FAIL · 0 no result** of 202.

Most important open findings

ID	Finding	Module	Severity	Status
`MINB-D01`	Send Template does not ask for header variables, so templates with a text-header {{1}} always fail	Inbox (WhatsApp chats)	High	Open
`MLGN-D04`	Native crash (SIGSEGV in RenderThread) seen once on an unstable emulator	Login (mobile sign-in)	Medium	Needs Decision
`MBRD-D01`	For a Tags audience, neither the review nor the send confirmation says how many contacts will receive the broadcast	Broadcasts (WhatsApp)	Medium	Open
`MINB-D02`	Location-header templates cannot be sent from the app: no location is asked for	Inbox (WhatsApp chats)	Medium	Open
`MINB-D03`	Authentication (one-time code) templates fail: the code button parameter is not sent	Inbox (WhatsApp chats)	Medium	Open

Latest runs

Run (reports/)	Started	Passed	Failed	Skipped	Not executed
`20261105-125911-single-MQTN-003`	5 Nov 2026	1	0	0	0
`20261105-124945-full`	5 Nov 2026	7	2	0	0
`20261105-124742-single-MQTN-001`	5 Nov 2026	1	0	0	0
`20261105-120411-full`	5 Nov 2026	7	1	0	0
`20261105-120242-single-MTKT-001`	5 Nov 2026	1	0	0	0
`20261105-114144-single-MDOC-006`	5 Nov 2026	1	0	0	0

Each run folder keeps one run, which can be a single test rather than a module regression.

Risks and next steps

- **High defects** - `MINB-D01`: agree fixes with development, then re-run the guard tests.
- **Medium defects** - `MLGN-D04`, `MBRD-D01`, `MINB-D02`, `MINB-D03`, `MAIA-D01`, `MLED-D01`: plan a fix and re-run the guard tests.
- **Single device** - all results come from one phone; a second device or Android version would catch device-specific problems.
- **Shared beta workspace** - tests read live data and the send test leaves real broadcasts; a dedicated test workspace would make results repeatable.

Appendix - for the QA and development team

Folder structure

Folder	Contents
`test/specs/smoke`	Smoke tests per module
`test/specs/regression`	Regression tests per module
`test/helpers`	Shared test steps: sign-in check, open a module, safety nets
`src/pages`	Page classes for each screen (selectors and actions)
`src/utils`, `src/data`, `src/config`	App lifecycle, adb and evidence helpers; test inputs; settings from config\.env
`config/`	WebdriverIO configuration, .env (local only) and .env.example
`scripts/`	PowerShell runners, emulator and APK scripts
`tools/excel`	Workbook generator; test cases and defects per module in modules/
`tools/docs`	Generator of this document
`docs/`	Test-case workbook, defect tracker, results ledger, this document, setup notes
`reports/<run>`	One folder per run: summary.md, Allure HTML, JUnit, screenshots, logs, failure diagnostics

Common commands

Command	What it does
`npm install`	Install dependencies (Appium: npm install -g appium; appium driver install uiautomator2)
`.\scripts\run-smoke.ps1`	Smoke tests of every module
`.\scripts\run-regression.ps1 -IncludeSmoke`	Every test
`.\scripts\run-regression.ps1 -IncludeSmoke -Grep MCMP`	One module by its ID prefix
`.\scripts\run-single-test.ps1 MBRD-012`	One test by its ID
`npm run excel`	Rebuild the workbooks and this document from the recorded results
`npm run docs:framework`	Rebuild this document (HTML and PDF); add -- -- watch to rebuild on every change

Conventions

- Test IDs are `<MODULE>-nnn` (`MAIA-nnn`, `MBRD-nnn`, `MCAL-nnn`, `MCMP-nnn`, `MCBT-nnn`, `MDOC-nnn`, `MINB-nnn`, `MLED-nnn`, `MLGN-nnn`, `REG-NAV-nnn`, `MORG-nnn`, `MPIP-nnn`, `MQTN-nnn`, `MTSK-nnn`, `MTPL-nnn`, `MTKT-nnn`, `MWLG-nnn`, `MWFL-nnn`); defect IDs are `<MODULE>-Dnn` (e.g. `MCMP-D02`).
- Selectors and expected texts live only in the page classes; the app has no test IDs, so accessibility labels are used.
- Test data uses recognisable names (`QA_Mobile_BC_`, `QA_Mobile_Draft_`, `qa_mobile_bc_allowlist_2026`) so it is never confused with real records.